

Experiment ‘p_5_5_repeat’ Results

June 4, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 1

Soft fails (resolution): 1

Verification attempts: 2

Problem Specification

Problem name: p_5_5_repeat

Natural language statement: Write a method that returns a string repeated n times. For example, repeat("ho", 3) returns "hohoho".

Method signature: p_5_5_repeat(str: string, n: int) returns (repeated_str: string)

Ensures

- $|repeated_str| == n * |str|$
- forall $i :: 0 \leq i < n \implies repeated_str[i * |str| .. (i + 1) * |str|] == str$

Requires

- $n \geq 0$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that returns a string repeated n times. For example, `repeat("ho", 3)`
↪ returns "hohoho".

The signature should be:

`p_5_5_repeat(str: string, n: int) returns (repeated_str: string)`

The method should respect the following contract:

requires `n >= 0`, ensures `|repeated_str| == n * |str|`, ensures forall `i :: 0 <= i < n ==>`
↪ `repeated_str[i * |str| .. (i + 1) * |str|] == str`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4OIZGqgCiFM2lkqZCUKKvdJ9CLR

Model: gpt-5.5-2026-04-23

Created at: 1780587351

Finish reason: stop

Usage: 378 tokens in, and 7061 tokens out

```
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var prev := repeated_str;
    var kk := k;

    assert |prev| == kk * |str|;
    forall i | 0 <= i < kk
      ensures prev[i * |str| .. (i + 1) * |str|] == str
    {
      assert 0 <= i < k;
      assert prev == repeated_str;
      assert repeated_str[i * |str| .. (i + 1) * |str|] == str;
    }
  }
}
```

```

repeated_str := prev + str;

assert repeated_str == prev + str;
assert |repeated_str| == |prev| + |str|;
assert (kk + 1) * |str| == kk * |str| + |str|;
assert |repeated_str| == (kk + 1) * |str|;
assert |prev| <= |repeated_str|;
assert repeated_str[..|prev|] == prev;
assert repeated_str[|prev|..|repeated_str|] == str;

forall i | 0 <= i < kk + 1
  ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
{
  var a := i * |str|;
  var b := (i + 1) * |str|;
  assert 0 <= |str|;
  assert 0 <= a <= b;
  if i < kk {
    assert i + 1 <= kk;
    assert b <= kk * |str|;
    assert b <= |prev|;
    assert repeated_str[a..b] == repeated_str[..|prev|][a..b];
    assert repeated_str[..|prev|][a..b] == prev[a..b];
    assert prev[a..b] == str;
  } else {
    assert i == kk;
    assert a == |prev|;
    assert b == |repeated_str|;
    assert repeated_str[a..b] == str;
  }
}

k := k + 1;
assert k == kk + 1;
}
assert k == n;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_5_repeat/program.dfy(1,7): Error: Verification of 'p_5_5_repeat' timed
  ↳ out after 31 seconds. (the limit can be increased using --verification-time-limit)
1 | method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
  | ~~~~~

```

Dafny program verifier finished with 1 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4SFaRN2bW7VIhu59rVjJG070Tl8

Model: gpt-5.5-2026-04-23

Created at: 1780587567

Finish reason: stop

Usage: 1151 tokens in, and 5784 tokens out

```
{
  repeated_str := "";
  var k := 0;

  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i {:trigger repeated_str[i * |str| .. (i + 1) * |str|]} ::
      0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var old := repeated_str;

    forall i | 0 <= i < k
      ensures old[i * |str| .. (i + 1) * |str|] == str
    {
      assert old == repeated_str;
      assert repeated_str[i * |str| .. (i + 1) * |str|] == str;
    }

    repeated_str := old + str;

    assert repeated_str == old + str;
    assert |old| == k * |str|;
    assert |repeated_str| == |old| + |str|;
    assert |repeated_str| == (k + 1) * |str|;
    assert repeated_str[..|old|] == old;
    assert repeated_str[|old|..] == str;

    forall i | 0 <= i < k + 1
      ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
    {
      var a := i * |str|;
      var b := (i + 1) * |str|;
      if i < k {
        assert i + 1 <= k;
        assert b <= k * |str|;
        assert b <= |old|;
      }
    }
  }
}
```

```

    assert repeated_str[a..b] == repeated_str[..|old|][a..b];
    assert repeated_str[..|old|][a..b] == old[a..b];
    assert old[a..b] == str;
  } else {
    assert i == k;
    assert a == |old|;
    assert b == |repeated_str|;
    assert repeated_str[a..b] == repeated_str[|old|..];
    assert repeated_str[|old|..] == str;
  }
}

k := k + 1;
}

assert k == n;
forall i | 0 <= i < n
  ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
{
  assert 0 <= i < k;
}
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_5_5_repeat/program.dfy(15,8): Error: invalid Ident
|
15 | var old := repeated_str;
| ^

problem_data/p_5_5_repeat/program.dfy(18,17): Error: openparen expected
|
18 | ensures old[i * |str| .. (i + 1) * |str|] == str
| ^

problem_data/p_5_5_repeat/program.dfy(18,28): Error: rbracket expected
|
18 | ensures old[i * |str| .. (i + 1) * |str|] == str
| ^

3 parse errors detected in program.dfy

```

Correct the error in code by altering only the method body. Produce and show only the
 → Dafny body, including the curly braces that surround it. Do not show the signature
 → nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4UON14nLPqE0Fevw3tGlZ97N1kd

Model: gpt-5.5-2026-04-23

Created at: 1780587700

Finish reason: stop

Usage: 2039 tokens in, and 4588 tokens out

```
{
  if n == 0 {
    repeated_str := "";
  } else {
    var prev := p_5_5_repeat(str, n - 1);
    repeated_str := prev + str;

    assert |prev| == (n - 1) * |str|;
    assert |repeated_str| == |prev| + |str|;
    assert n * |str| == (n - 1) * |str| + |str|;
    assert |repeated_str| == n * |str|;

    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..] == str;

    forall i | 0 <= i < n
      ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
    {
      var a := i * |str|;
      var b := (i + 1) * |str|;
      assert 0 <= |str|;
      assert 0 <= a <= b;

      if i < n - 1 {
        assert 0 <= i < n - 1;
        assert i + 1 <= n - 1;
        assert b <= (n - 1) * |str|;
        assert b <= |prev|;
        assert repeated_str[a..b] == repeated_str[..|prev|][a..b];
        assert repeated_str[..|prev|][a..b] == prev[a..b];
        assert prev[a..b] == str;
      } else {
        assert i == n - 1;
        assert a == |prev|;
        assert b == |repeated_str|;
        assert repeated_str[a..b] == repeated_str[|prev|..];
        assert repeated_str[|prev|..] == str;
      }
    }
  }
}
```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_5_5_repeat/program.dfy(21,26): Error: upper bound below lower bound or  
  ↪ above length of sequence
```

```
|  
21 | ensures repeated_str[i * |str| .. (i + 1) * |str|] == str  
| ^
```

```
problem_data/p_5_5_repeat/program.dfy(31,8): Error: assertion might not hold
```

```
|  
31 | assert b <= (n - 1) * |str|;  
| ~~~~~
```

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4WyURPTExJZm7yx6kES3G8JEVNY

Model: gpt-5.5-2026-04-23

Created at: 1780587860

Finish reason: stop

Usage: 2654 tokens in, and 8180 tokens out

```
{  
  if n == 0 {  
    repeated_str := "";  
    assert |repeated_str| == 0;  
    assert 0 * |str| == 0;  
  } else {  
    var L := |str|;  
    var m := n - 1;  
    assert 0 <= m;  
    assert n == m + 1;  
  
    var prev := p_5_5_repeat(str, m);  
    repeated_str := prev + str;  
  
    assert |prev| == m * L;  
    assert |repeated_str| == |prev| + L;  
    assert m * L + L == (m + 1) * L;  
    assert |repeated_str| == (m + 1) * L;  
    assert |repeated_str| == n * L;  
    assert repeated_str[..|prev|] == prev;  
    assert repeated_str[|prev|..|repeated_str|] == str;
```

```

forall i | 0 <= i < n
  ensures 0 <= i * L && i * L <= (i + 1) * L && (i + 1) * L <= |repeated_str| &&
    repeated_str[i * L .. (i + 1) * L] == str
{
  assert 0 <= L;
  assert 0 <= i;
  assert i + 1 <= n;
  assert 0 <= n - (i + 1);
  assert 0 <= i * L;
  assert (i + 1) * L == i * L + L;
  assert i * L <= (i + 1) * L;
  assert n * L == (i + 1) * L + (n - (i + 1)) * L;
  assert 0 <= (n - (i + 1)) * L;
  assert (i + 1) * L <= n * L;
  assert (i + 1) * L <= |repeated_str|;

  if i < m {
    assert 0 <= i < m;
    assert i + 1 <= m;
    assert 0 <= m - (i + 1);
    assert m * L == (i + 1) * L + (m - (i + 1)) * L;
    assert 0 <= (m - (i + 1)) * L;
    assert (i + 1) * L <= m * L;
    assert (i + 1) * L <= |prev|;
    assert prev[i * L .. (i + 1) * L] == str;
    assert repeated_str[i * L .. (i + 1) * L] == repeated_str[..|prev|][i * L .. (i +
      ↪ 1) * L];
    assert repeated_str[..|prev|][i * L .. (i + 1) * L] == prev[i * L .. (i + 1) * L];
    assert repeated_str[i * L .. (i + 1) * L] == str;
  } else {
    assert i == m;
    assert i * L == m * L;
    assert i * L == |prev|;
    assert i + 1 == m + 1;
    assert (i + 1) * L == (m + 1) * L;
    assert (i + 1) * L == |repeated_str|;
    assert repeated_str[i * L .. (i + 1) * L] == repeated_str[|prev|..|repeated_str|];
    assert repeated_str[i * L .. (i + 1) * L] == str;
  }
}

assert L == |str|;
assert |repeated_str| == n * |str|;
assert forall i :: 0 <= i < n ==>
  (0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |repeated_str|
    ↪ &&
    repeated_str[i * |str| .. (i + 1) * |str|] == str);
}
}

```

Final Program

```
method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
  requires n >= 0
  ensures |repeated_str| == n * |str|
  ensures forall i :: 0 <= i < n ==> repeated_str[i * |str| .. (i + 1) * |str| ] ==
    ↪ str
{
  if n == 0 {
    repeated_str := "";
    assert |repeated_str| == 0;
    assert 0 * |str| == 0;
  } else {
    var L := |str|;
    var m := n - 1;
    assert 0 <= m;
    assert n == m + 1;

    var prev := p_5_5_repeat(str, m);
    repeated_str := prev + str;

    assert |prev| == m * L;
    assert |repeated_str| == |prev| + L;
    assert m * L + L == (m + 1) * L;
    assert |repeated_str| == (m + 1) * L;
    assert |repeated_str| == n * L;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..|repeated_str|] == str;

    forall i | 0 <= i < n
      ensures 0 <= i * L && i * L <= (i + 1) * L && (i + 1) * L <= |repeated_str| &&
        repeated_str[i * L .. (i + 1) * L] == str
    {
      assert 0 <= L;
      assert 0 <= i;
      assert i + 1 <= n;
      assert 0 <= n - (i + 1);
      assert 0 <= i * L;
      assert (i + 1) * L == i * L + L;
      assert i * L <= (i + 1) * L;
      assert n * L == (i + 1) * L + (n - (i + 1)) * L;
      assert 0 <= (n - (i + 1)) * L;
      assert (i + 1) * L <= n * L;
      assert (i + 1) * L <= |repeated_str|;

      if i < m {
        assert 0 <= i < m;
        assert i + 1 <= m;
        assert 0 <= m - (i + 1);
        assert m * L == (i + 1) * L + (m - (i + 1)) * L;
        assert 0 <= (m - (i + 1)) * L;
        assert (i + 1) * L <= m * L;
      }
    }
  }
}
```

```

    assert (i + 1) * L <= |prev|;
    assert prev[i * L .. (i + 1) * L] == str;
    assert repeated_str[i * L .. (i + 1) * L] == repeated_str[..|prev|][i * L .. (i +
    ↪ 1) * L];
    assert repeated_str[..|prev|][i * L .. (i + 1) * L] == prev[i * L .. (i + 1) * L];
    assert repeated_str[i * L .. (i + 1) * L] == str;
} else {
    assert i == m;
    assert i * L == m * L;
    assert i * L == |prev|;
    assert i + 1 == m + 1;
    assert (i + 1) * L == (m + 1) * L;
    assert (i + 1) * L == |repeated_str|;
    assert repeated_str[i * L .. (i + 1) * L] == repeated_str[|prev|..|repeated_str|];
    assert repeated_str[i * L .. (i + 1) * L] == str;
}
}

assert L == |str|;
assert |repeated_str| == n * |str|;
assert forall i :: 0 <= i < n ==>
    (0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |repeated_str|
    ↪ &&
    repeated_str[i * |str| .. (i + 1) * |str|] == str);
}
}

```

Total Token Usage

Input tokens: 6222

Output tokens: 25613

Reasoning tokens: 22728

Sum of 'total tokens': 31835

Experiment Timings

Overall Experiment started at 1780587337781, ended at 1780588035777, lasting 697996ms (698.00 seconds)

Iteration #4 started at 1780587859090, ended at 1780588035766, lasting 176676ms (176.68 seconds)

Iteration #1 started at 1780587337810, ended at 1780587565286, lasting 227476ms (227.48 seconds)

Iteration #2 started at 1780587565290, ended at 1780587699396, lasting 134106ms (134.11 seconds)

Iteration #3 started at 1780587699397, ended at 1780587859087, lasting 159690ms (159.69 seconds)

Experiment 'p_5_9_f_cone_surface' Results

June 4, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 4

Problem Specification

Problem name: p_5_9_f_cone_surface

Natural language statement: Write a method that computes the surface area of a cone with a circular base with a residual error less than 0.01, given radius and height.

Method signature: p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)

Ensures

- $-0.01 < (((\text{surface} / (3.14159 * r)) - r) * ((\text{surface} / (3.14159 * r)) - r)) - ((h * h) + (r * r)) < 0.01$

Requires

- $r > 0.0$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that computes the surface area of a cone with a circular base with a
→ residual error less than 0.01, given radius and height.

The signature should be:

p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)

The method should respect the following contract:

requires $r > 0.0$, ensures $-0.01 < (((\text{surface} / (3.14159 * r)) - r) * ((\text{surface} / (3.14159 * r)) - r)) - ((h * h) + (r * r)) < 0.01$
→

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4aC3lYb4JcqqAJI8JyVT9eAtrXW

Model: gpt-5.5-2026-04-23

Created at: 1780588060

Finish reason: stop

Usage: 406 tokens in, and 6060 tokens out

```
{
  assert 3.14159 > 0.0;
  assert 3.14159 * r > 0.0;
  assert h * h >= 0.0;
  assert r * r > 0.0;
  assert h * h + r * r > 0.0;

  var root: real :| root * root == h * h + r * r;
  var slant := if root >= 0.0 then root else -root;
  assert slant * slant == h * h + r * r;

  surface := 3.14159 * r * (r + slant);

  assert 3.14159 * r != 0.0;
  assert surface / (3.14159 * r) == r + slant;
  assert (surface / (3.14159 * r)) - r == slant;
  assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) == h * h + r * r;
  → r;
```

```

assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
    ↪ (r * r)) == 0.0;
assert -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((
    ↪ h * h) + (r * r)) < 0.01;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_9_f_cone_surface/program.dfy(11,17): Error: cannot establish the
    ↪ existence of LHS values that satisfy the such-that predicate. Note, no trigger was
    ↪ found for the such-that predicate, which may be the reason the proof failed. To
    ↪ give a trigger explicitly, use the {:trigger} attribute. For more information, see
    ↪ the section on quantifier instantiation rules in the reference manual.

```

```

|
11 | var root: real :| root * root == h * h + r * r;
| ~~~~~

```

```

problem_data/p_5_9_f_cone_surface/program.dfy(18,2): Error: assertion might not hold

```

```

|
18 | assert surface / (3.14159 * r) == r + slant;
| ~~~~~

```

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4cwzGhTsatKsKC8qIhIb76Yu739

Model: gpt-5.5-2026-04-23

Created at: 1780588230

Finish reason: stop

Usage: 1110 tokens in, and 10316 tokens out

```

{
  var y := h * h + r * r;
  assert h * h >= 0.0;
  assert r * r > 0.0;
  assert y > 0.0;

  var x := if y >= 1.0 then y else 1.0;
  if y >= 1.0 {
    assert x == y;
    assert y * y >= y;
    assert x * x >= y;
  }
}

```

```

    assert x > 0.0;
} else {
    assert x == 1.0;
    assert x * x == 1.0;
    assert x * x >= y;
    assert x > 0.0;
}

while x * x - y >= 0.01
    invariant y > 0.0
    invariant x > 0.0
    invariant x * x >= y
    decreases 100000.0 * (x * x - y)
{
    var oldX := x;
    var e := oldX * oldX - y;
    assert e >= 0.01;
    assert e > 0.0;
    assert oldX > 0.0;
    assert oldX * oldX > y;
    assert e < oldX * oldX;
    assert 2.0 * oldX > 0.0;

    var d := e / (2.0 * oldX);
    assert d > 0.0;
    assert 2.0 * oldX * d == e;
    assert d < oldX / 2.0;
    assert d * d < d * oldX / 2.0;
    assert d * oldX / 2.0 == e / 4.0;
    assert d * d < e / 4.0;

    x := oldX - d;
    assert x > 0.0;
    calc {
        x * x - y;
    ==
        (oldX - d) * (oldX - d) - y;
    ==
        oldX * oldX - 2.0 * oldX * d + d * d - y;
    ==
        e - 2.0 * oldX * d + d * d;
    ==
        d * d;
    }
    assert x * x - y == d * d;
    assert x * x >= y;
    assert x * x - y < e / 4.0;
    assert 100000.0 * (x * x - y) < 25000.0 * e;
    assert 25000.0 * e <= 100000.0 * e - 1.0;
    assert 100000.0 * (x * x - y) < 100000.0 * e - 1.0;
}

```

```

assert x * x - y >= 0.0;
assert x * x - y < 0.01;

var denom := 3.14159 * r;
assert 3.14159 > 0.0;
assert denom > 0.0;
assert denom != 0.0;

surface := (r + x) * denom;
assert surface == (r + x) * denom;
assert surface == denom * (r + x);
assert surface / denom == r + x;
assert denom == 3.14159 * r;
assert surface / (3.14159 * r) == r + x;
assert (surface / (3.14159 * r)) - r == x;
assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == x * x;
assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
    ↪ (r * r)) == x * x - y;
assert -0.01 < x * x - y;
assert x * x - y < 0.01;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_9_f_cone_surface/program.dfy(8,2): Error: assertion might not hold
|
8 | assert y > 0.0;
|   ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4hHU6j4r5w9qPMdAURfwuvRUG7T

Model: gpt-5.5-2026-04-23

Created at: 1780588499

Finish reason: stop

Usage: 2076 tokens in, and 5483 tokens out

```

{
  var y := h * h + r * r;
  assert h * h >= 0.0;
}

```

```

assert r * r > 0.0;
assert h * h + r * r > 0.0;
assert y == h * h + r * r;
assert y > 0.0;

var x := if y >= 1.0 then y else 1.0;
if y >= 1.0 {
  assert x == y;
  assert y * y >= y;
  assert x * x >= y;
  assert x > 0.0;
} else {
  assert x == 1.0;
  assert x * x == 1.0;
  assert x * x >= y;
  assert x > 0.0;
}

while x * x - y >= 0.01
  invariant y > 0.0
  invariant x > 0.0
  invariant x * x >= y
  decreases 100000.0 * (x * x - y)
{
  var oldX := x;
  var e := oldX * oldX - y;
  assert e >= 0.01;
  assert e > 0.0;
  assert oldX > 0.0;
  assert oldX * oldX >= y;
  assert e == oldX * oldX - y;
  assert e < oldX * oldX;
  assert 2.0 * oldX > 0.0;

  var d := e / (2.0 * oldX);
  assert d > 0.0;
  assert 2.0 * oldX * d == e;
  assert d < oldX / 2.0;
  assert d * d < d * oldX / 2.0;
  assert d * oldX / 2.0 == e / 4.0;
  assert d * d < e / 4.0;

  x := oldX - d;
  assert x > 0.0;
  calc {
    x * x - y;
  ==
    (oldX - d) * (oldX - d) - y;
  ==
    oldX * oldX - 2.0 * oldX * d + d * d - y;
  ==

```

```

    e - 2.0 * oldX * d + d * d;
  ==
    d * d;
}
assert x * x - y == d * d;
assert x * x >= y;
assert x * x - y < e / 4.0;
assert 100000.0 * (x * x - y) < 25000.0 * e;
assert 25000.0 * e <= 100000.0 * e - 1.0;
assert 100000.0 * (x * x - y) < 100000.0 * e - 1.0;
}

assert x * x - y >= 0.0;
assert x * x - y < 0.01;

var denom := 3.14159 * r;
assert 3.14159 > 0.0;
assert denom > 0.0;
assert denom != 0.0;

surface := denom * (r + x);
assert surface == denom * (r + x);
assert surface / denom == r + x;
assert denom == 3.14159 * r;
assert surface / (3.14159 * r) == r + x;
assert (surface / (3.14159 * r)) - r == x;
assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == x * x;
assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
    ↪ (r * r)) == x * x - y;
assert -0.01 < x * x - y;
assert x * x - y < 0.01;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_9_f_cone_surface/program.dfy(10,2): Error: assertion might not hold
|
10 | assert y > 0.0;
    | ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4jeQThukPio5AsH8GEUBUKvMe7J

Model: gpt-5.5-2026-04-23

Created at: 1780588646

Finish reason: stop

Usage: 3200 tokens in, and 2056 tokens out

```
{
  var hh := h * h;
  var rr := r * r;
  assert hh >= 0.0;
  assert rr > 0.0;
  var y := hh + rr;
  assert y >= rr;
  assert y > 0.0;
  assert y == h * h + r * r;

  var x := if y >= 1.0 then y else 1.0;
  if y >= 1.0 {
    assert x == y;
    assert y > 0.0;
    assert y * y >= y;
    assert x * x >= y;
    assert x > 0.0;
  } else {
    assert x == 1.0;
    assert y < 1.0;
    assert x * x == 1.0;
    assert x * x >= y;
    assert x > 0.0;
  }

  while x * x - y >= 0.01
    invariant y > 0.0
    invariant y == h * h + r * r
    invariant x > 0.0
    invariant x * x >= y
    decreases 100000.0 * (x * x - y)
  {
    var oldX := x;
    var e := oldX * oldX - y;
    assert e >= 0.01;
    assert e > 0.0;
    assert oldX > 0.0;
    assert oldX * oldX >= y;
    assert e == oldX * oldX - y;
    assert e < oldX * oldX;
    assert 2.0 * oldX > 0.0;

    var d := e / (2.0 * oldX);
```

```

assert d > 0.0;
assert 2.0 * oldX * d == e;
assert d < oldX / 2.0;
assert d * d < d * oldX / 2.0;
assert d * oldX / 2.0 == e / 4.0;
assert d * d < e / 4.0;

x := oldX - d;
assert x > 0.0;
calc {
  x * x - y;
==
  (oldX - d) * (oldX - d) - y;
==
  oldX * oldX - 2.0 * oldX * d + d * d - y;
==
  e - 2.0 * oldX * d + d * d;
==
  d * d;
}
assert x * x - y == d * d;
assert x * x >= y;
assert x * x - y < e / 4.0;
assert 100000.0 * (x * x - y) < 25000.0 * e;
assert 25000.0 * e <= 100000.0 * e - 1.0;
assert 100000.0 * (x * x - y) < 100000.0 * e - 1.0;
}

assert x * x - y >= 0.0;
assert x * x - y < 0.01;

var denom := 3.14159 * r;
assert 3.14159 > 0.0;
assert denom > 0.0;
assert denom != 0.0;

surface := denom * (r + x);
assert surface == denom * (r + x);
assert surface / denom == r + x;
assert denom == 3.14159 * r;
assert surface / (3.14159 * r) == r + x;
assert (surface / (3.14159 * r)) - r == x;
assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == x * x;
assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
  ↪ (r * r)) == x * x - y;
assert -0.01 < x * x - y;
assert x * x - y < 0.01;
}

```

Final Program

```
method p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)
  requires r > 0.0
  ensures -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)
    ↪ ) - ((h * h) + (r * r)) < 0.01
{
  var hh := h * h;
  var rr := r * r;
  assert hh >= 0.0;
  assert rr > 0.0;
  var y := hh + rr;
  assert y >= rr;
  assert y > 0.0;
  assert y == h * h + r * r;

  var x := if y >= 1.0 then y else 1.0;
  if y >= 1.0 {
    assert x == y;
    assert y > 0.0;
    assert y * y >= y;
    assert x * x >= y;
    assert x > 0.0;
  } else {
    assert x == 1.0;
    assert y < 1.0;
    assert x * x == 1.0;
    assert x * x >= y;
    assert x > 0.0;
  }

  while x * x - y >= 0.01
    invariant y > 0.0
    invariant y == h * h + r * r
    invariant x > 0.0
    invariant x * x >= y
    decreases 100000.0 * (x * x - y)
  {
    var oldX := x;
    var e := oldX * oldX - y;
    assert e >= 0.01;
    assert e > 0.0;
    assert oldX > 0.0;
    assert oldX * oldX >= y;
    assert e == oldX * oldX - y;
    assert e < oldX * oldX;
    assert 2.0 * oldX > 0.0;

    var d := e / (2.0 * oldX);
    assert d > 0.0;
    assert 2.0 * oldX * d == e;
    assert d < oldX / 2.0;
```

```

assert d * d < d * oldX / 2.0;
assert d * oldX / 2.0 == e / 4.0;
assert d * d < e / 4.0;

x := oldX - d;
assert x > 0.0;
calc {
  x * x - y;
==
  (oldX - d) * (oldX - d) - y;
==
  oldX * oldX - 2.0 * oldX * d + d * d - y;
==
  e - 2.0 * oldX * d + d * d;
==
  d * d;
}
assert x * x - y == d * d;
assert x * x >= y;
assert x * x - y < e / 4.0;
assert 100000.0 * (x * x - y) < 25000.0 * e;
assert 25000.0 * e <= 100000.0 * e - 1.0;
assert 100000.0 * (x * x - y) < 100000.0 * e - 1.0;
}

assert x * x - y >= 0.0;
assert x * x - y < 0.01;

var denom := 3.14159 * r;
assert 3.14159 > 0.0;
assert denom > 0.0;
assert denom != 0.0;

surface := denom * (r + x);
assert surface == denom * (r + x);
assert surface / denom == r + x;
assert denom == 3.14159 * r;
assert surface / (3.14159 * r) == r + x;
assert (surface / (3.14159 * r)) - r == x;
assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == x * x;
assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
  ↪ (r * r)) == x * x - y;
assert -0.01 < x * x - y;
assert x * x - y < 0.01;
}

```

Total Token Usage

Input tokens: 6792

Output tokens: 23915

Reasoning tokens: 20339
Sum of ‘total tokens’: 30707

Experiment Timings

Overall Experiment started at 1780588060230, ended at 1780588698267, lasting 638037ms (638.04 seconds)
Iteration #4 started at 1780588644735, ended at 1780588698256, lasting 53521ms (53.52 seconds)
Iteration #1 started at 1780588060230, ended at 1780588226736, lasting 166506ms (166.51 seconds)
Iteration #2 started at 1780588226741, ended at 1780588498896, lasting 272155ms (272.16 seconds)
Iteration #3 started at 1780588498896, ended at 1780588644735, lasting 145839ms (145.84 seconds)

Experiment 'p_5_1_a_smallest' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_1_a_smallest

Natural language statement: Write a method that returns the smallest of the arguments.

Method signature: p_5_1_a_smallest(x: real, y: real, z: real) returns (smallest: real)

Ensures

- `smallest <= x && smallest <= y && smallest <= z`
- `smallest == x || smallest == y || smallest == z`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that returns the smallest of the arguments.

The signature should be:

`p_5_1_a_smallest(x: real, y: real, z: real) returns (smallest: real)`

The method should respect the following contract:

```
ensures smallest <= x && smallest <= y && smallest <= z, ensures smallest == x ||  
    ↪ smallest == y || smallest == z
```

Produce and show only the Dafny body of this method, including the curly braces that
 ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 ↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmyJFLzowiW69xy3ujYM6XaAAmMGE

Model: gpt-5.5-2026-04-23

Created at: 1780563945

Finish reason: stop

Usage: 306 tokens in, and 558 tokens out

```
{  
  if x <= y {  
    if x <= z {  
      smallest := x;  
    } else {  
      smallest := z;  
    }  
  } else {  
    if y <= z {  
      smallest := y;  
    } else {  
      smallest := z;  
    }  
  }  
}
```

Final Program

```
method p_5_1_a_smallest(x: real, y: real, z: real) returns (smallest: real)
  ensures smallest <= x && smallest <= y && smallest <= z
  ensures smallest == x || smallest == y || smallest == z
{
  if x <= y {
    if x <= z {
      smallest := x;
    } else {
      smallest := z;
    }
  } else {
    if y <= z {
      smallest := y;
    } else {
      smallest := z;
    }
  }
}
```

Total Token Usage

Input tokens: 306

Output tokens: 558

Reasoning tokens: 469

Sum of ‘total tokens’: 864

Experiment Timings

Overall Experiment started at 1780563930267, ended at 1780564016856, lasting 86589ms (86.59 seconds)

Iteration #1 started at 1780563930277, ended at 1780564016755, lasting 86478ms (86.48 seconds)

Experiment 'p_5_1_b_average' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_1_b_average

Natural language statement: Write a method that returns the average of the arguments.

Method signature: p_5_1_b_average(x: real, y: real, z: real) returns (average: real)

Ensures

- `average == (x + y + z) / 3.0`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that returns the average of the arguments.

The signature should be:

`p_5_1_b_average(x: real, y: real, z: real) returns (average: real)`

The method should respect the following contract:

`ensures average == (x + y + z) / 3.0`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmyKqB8bEoOw6W3BvBA0lFkSNAB3s

Model: gpt-5.5-2026-04-23

Created at: 1780564044

Finish reason: stop

Usage: 282 tokens in, and 63 tokens out

```
{  
  average := (x + y + z) / 3.0;  
}
```

Final Program

```
method p_5_1_b_average(x: real, y: real, z: real) returns (average: real)  
  ensures average == (x + y + z) / 3.0  
{  
  average := (x + y + z) / 3.0;  
}
```

Total Token Usage

Input tokens: 282

Output tokens: 63

Reasoning tokens: 26

Sum of ‘total tokens’: 345

Experiment Timings

Overall Experiment started at 1780564041692, ended at 1780564076798, lasting 35106ms (35.11 seconds)

Iteration #1 started at 1780564041797, ended at 1780564076795, lasting 34998ms (35.00 seconds)

Experiment 'p_5_2_a_all_the_same' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_2_a_all_the_same

Natural language statement: Write a method returning true if the arguments are all the same.

Method signature: p_5_2_a_all_the_same(x: real, y: real, z: real) returns (all_same: bool)

Ensures

- all_same == (x == y && y == z)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method returning true if the arguments are all the same.

The signature should be:

`p_5_2_a_all_the_same(x: real, y: real, z: real) returns (all_same: bool)`

The method should respect the following contract:

`ensures all_same == (x == y && y == z)`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmyLXKUfqs8SZa4oa2z9rCTG5erZC

Model: gpt-5.5-2026-04-23

Created at: 1780564087

Finish reason: stop

Usage: 286 tokens in, and 27 tokens out

```
{  
  all_same := x == y && y == z;  
}
```

Final Program

```
method p_5_2_a_all_the_same(x: real, y: real, z: real) returns (all_same: bool)  
  ensures all_same == (x == y && y == z)  
{  
  all_same := x == y && y == z;  
}
```

Total Token Usage

Input tokens: 286

Output tokens: 27

Reasoning tokens: 0

Sum of ‘total tokens’: 313

Experiment Timings

Overall Experiment started at 1780564087414, ended at 1780564126567, lasting 39153ms (39.15 seconds)

Iteration #1 started at 1780564087415, ended at 1780564126566, lasting 39151ms (39.15 seconds)

Experiment 'p_5_2_b_all_different' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_2_b_all_different

Natural language statement: Write a method returning true if the arguments are all different.

Method signature: p_5_2_b_all_different(x: real, y: real, z: real) returns (all_diff: bool)

Ensures

- `all_diff == (x != y && y != z && x != z)`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method returning true if the arguments are all different.

The signature should be:

`p_5_2_b_all_different(x: real, y: real, z: real) returns (all_diff: bool)`

The method should respect the following contract:

`ensures all_diff == (x != y && y != z && x != z)`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmyMDytizrvIKHLpPTm7Cv6uvOaJs

Model: gpt-5.5-2026-04-23

Created at: 1780564129

Finish reason: stop

Usage: 292 tokens in, and 63 tokens out

```
{  
  all_diff := x != y && y != z && x != z;  
}
```

Final Program

```
method p_5_2_b_all_different(x: real, y: real, z: real) returns (all_diff: bool)  
  ensures all_diff == (x != y && y != z && x != z)  
{  
  all_diff := x != y && y != z && x != z;  
}
```

Total Token Usage

Input tokens: 292

Output tokens: 63

Reasoning tokens: 26

Sum of ‘total tokens’: 355

Experiment Timings

Overall Experiment started at 1780564128867, ended at 1780564154394, lasting 25527ms (25.53 seconds)

Iteration #1 started at 1780564128867, ended at 1780564154394, lasting 25527ms (25.53 seconds)

Experiment ‘p_5_2_c_sorted’ Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_2_c_sorted

Natural language statement: Write a method returning true if the arguments are sorted, with the smallest one coming first.

Method signature: p_5_2_c_sorted(x: real, y: real, z: real) returns (is_sorted: bool)

Ensures

- `is_sorted == (x <= y && y <= z)`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method returning true if the arguments are sorted, with the smallest one coming
→ first.

The signature should be:

p_5_2_c_sorted(x: real, y: real, z: real) returns (is_sorted: bool)

The method should respect the following contract:

ensures is_sorted == (x <= y && y <= z)

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmyMgYNY2dTvMrHXxwtQM1pUIHxhp

Model: gpt-5.5-2026-04-23

Created at: 1780564158

Finish reason: stop

Usage: 292 tokens in, and 27 tokens out

```
{  
  is_sorted := x <= y && y <= z;  
}
```

Final Program

```
method p_5_2_c_sorted(x: real, y: real, z: real) returns (is_sorted: bool)  
  ensures is_sorted == (x <= y && y <= z)  
{  
  is_sorted := x <= y && y <= z;  
}
```

Total Token Usage

Input tokens: 292

Output tokens: 27

Reasoning tokens: 0

Sum of ‘total tokens’: 319

Experiment Timings

Overall Experiment started at 1780564157760, ended at 1780564179120, lasting 21360ms (21.36 seconds)

Iteration #1 started at 1780564157766, ended at 1780564179120, lasting 21354ms (21.35 seconds)

Experiment 'p_5_3_first_digit' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_3_first_digit

Natural language statement: Write a method returning the first digit of the argument.

Method signature: p_5_3_first_digit(n: int) returns (first: int)

Ensures

- `0 <= first < 10`
- `intToSeq(n)[0] == first`

Requires

- `0 <= n`

Functional Code Given

```
function intToSeq(n: int): seq<int>
  requires n >= 0
  ensures forall d :: d in intToSeq(n) ==> 0 <= d < 10
{
  if n < 10 then [n]
  else intToSeq(n / 10) + [n % 10]
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method returning the first digit of the argument.

The signature should be:

`p_5_3_first_digit(n: int)` returns `(first: int)`

The method should respect the following contract:

`requires 0 <= n, ensures 0 <= first < 10, ensures intToSeq(n)[0] == first`

The contract uses the following dafny code:

```
function intToSeq(n: int): seq<int>
  requires n >= 0
  ensures forall d :: d in intToSeq(n) ==> 0 <= d < 10
{
  if n < 10 then [n]
  else intToSeq(n / 10) + [n % 10]
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmyN5AcD6QQIDqE2475O3vkhcv9yf

Model: gpt-5.5-2026-04-23

Created at: 1780564183

Finish reason: stop

Usage: 468 tokens in, and 1088 tokens out

```
{
  if n < 10 {
    first := n;
  } else {
    first := p_5_3_first_digit(n / 10);
  }
}
```

Final Program

```
function intToSeq(n: int): seq<int>
  requires n >= 0
  ensures forall d :: d in intToSeq(n) ==> 0 <= d < 10
{
  if n < 10 then [n]
  else intToSeq(n / 10) + [n % 10]
}

method p_5_3_first_digit(n: int) returns (first: int)
  requires 0 <= n
  ensures 0 <= first < 10
  ensures intToSeq(n)[0] == first
{
  if n < 10 {
    first := n;
  } else {
    first := p_5_3_first_digit(n / 10);
  }
}
```

Total Token Usage

Input tokens: 468

Output tokens: 1088

Reasoning tokens: 1023

Sum of 'total tokens': 1556

Experiment Timings

Overall Experiment started at 1780564182758, ended at 1780564244618, lasting 61860ms (61.86 seconds)

Iteration #1 started at 1780564182764, ended at 1780564244618, lasting 61854ms (61.85 seconds)

Experiment 'p_5_4_middle' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_4_middle

Natural language statement: Write a method that returns the middle character of a string if the length is odd, or the two middle characters if the length is even. For example, middle("middle") returns "dd".

Method signature: p_5_4_middle(str: string) returns (middle_str: string)

Ensures

- if |str| % 2 == 1 then |middle_str| == 1 else |middle_str| == 2
- if |str| % 2 == 1 then middle_str[0] == str[|str| / 2] else middle_str == str[|str| / 2 - 1 .. (|str| / 2) + 1]

Requires

- 0 < |str|

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that returns the middle character of a string if the length is odd, or the
↪ two middle characters if the length is even. For example, `middle("middle")`
↪ returns `"dd"`.

The signature should be:

`p_5_4_middle(str: string)` returns `(middle_str: string)`

The method should respect the following contract:

requires $0 < |str|$, ensures if $|str| \% 2 == 1$ then $|middle_str| == 1$ else $|middle_str| ==$
↪ 2 , ensures if $|str| \% 2 == 1$ then $middle_str[0] == str[|str| / 2]$ else $middle_str$
↪ $== str[|str| / 2 - 1 .. (|str| / 2) + 1]$

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnSBzakaqzyL7SvcojBSjJmnHObh8

Model: gpt-5.5-2026-04-23

Created at: 1780678815

Finish reason: stop

Usage: 470 tokens in, and 1224 tokens out

```
{
  var n := |str|;
  var mid := n / 2;

  if n % 2 == 1 {
    assert n == 2 * mid + 1;
    assert mid < n;
    assert mid + 1 <= n;
    middle_str := str[mid .. mid + 1];
    assert |middle_str| == 1;
    assert middle_str[0] == str[mid];
  } else {
    assert n % 2 == 0;
    assert n == 2 * mid;
    assert mid > 0;
    assert mid + 1 <= n;
    middle_str := str[mid - 1 .. mid + 1];
  }
```

```
    assert |middle_str| == 2;
  }
}
```

Final Program

```
method p_5_4_middle(str: string) returns (middle_str: string)
  requires 0 < |str|
  ensures if |str| % 2 == 1 then |middle_str| == 1 else |middle_str| == 2
  ensures if |str| % 2 == 1 then middle_str[0] == str[|str| / 2] else middle_str ==
    ↪ str[|str| / 2 - 1 .. (|str| / 2) + 1]
{
  var n := |str|;
  var mid := n / 2;

  if n % 2 == 1 {
    assert n == 2 * mid + 1;
    assert mid < n;
    assert mid + 1 <= n;
    middle_str := str[mid .. mid + 1];
    assert |middle_str| == 1;
    assert middle_str[0] == str[mid];
  } else {
    assert n % 2 == 0;
    assert n == 2 * mid;
    assert mid > 0;
    assert mid + 1 <= n;
    middle_str := str[mid - 1 .. mid + 1];
    assert |middle_str| == 2;
  }
}
```

Total Token Usage

Input tokens: 470

Output tokens: 1224

Reasoning tokens: 1024

Sum of 'total tokens': 1694

Experiment Timings

Overall Experiment started at 1780678795916, ended at 1780678910818, lasting 114902ms (114.90 seconds)

Iteration #1 started at 1780678795919, ended at 1780678910814, lasting 114895ms (114.90 seconds)

Experiment ‘p_5_7_count_words’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_7_count_words

Natural language statement: Write a method that returns the count of all words in a string. Words are separated by spaces. For example, countWords("Mary had a little lamb") should return 5.

Method signature: p_5_7_count_words(str: string) returns (word_count: int)

Ensures

- word_count == count_words(str)

Functional Code Given

```
function remove_leading_spaces(s: string): (result: string)
  ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] == ' ' then remove_leading_spaces(s[1..])
  else s
}

function remove_word(s: string): (result: string)
  ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] != ' ' then remove_word(s[1..])
  else s
}

function count_words(s: string): int
  decreases |s|
{
```

```
var s2 := remove_leading_spaces(s);  
if |s2| > 0 then 1 + count_words(remove_word(s2))  
else 0  
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that returns the count of all words in a string. Words are separated by
→ spaces. For example, `countWords("Mary had a little lamb")` should return 5.

The signature should be:

`p_5_7_count_words(str: string)` returns (`word_count: int`)

The method should respect the following contract:

`ensures word_count == count_words(str)`

The contract uses the following dafny code:

```
function remove_leading_spaces(s: string): (result: string)
  ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] == ' ' then remove_leading_spaces(s[1..])
  else s
}

function remove_word(s: string): (result: string)
  ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] != ' ' then remove_word(s[1..])
  else s
}

function count_words(s: string): int
  decreases |s|
{
  var s2 := remove_leading_spaces(s);
  if |s2| > 0 then 1 + count_words(remove_word(s2))
  else 0
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnSDnc5sEvKOBMudbkFB1U2Md3VdY

Model: gpt-5.5-2026-04-23

Created at: 1780678927

Finish reason: stop

Usage: 748 tokens in, and 3587 tokens out

```
{
  if |str| == 0 {
    word_count := 0;
    assert remove_leading_spaces(str) == str;
    assert count_words(str) == 0;
  } else if str[0] == ' ' {
    var rest := p_5_7_count_words(str[1..]);
    word_count := rest;
    assert str[1..] == str[1..];
    assert remove_leading_spaces(str) == remove_leading_spaces(str[1..]);
    assert count_words(str) == count_words(str[1..]);
  } else {
    var i := 0;
    while i < |str| && str[i] != ' '
      invariant 0 <= i <= |str|
      invariant remove_word(str) == remove_word(str[i..])
    {
      assert |str[i..]| > 0;
      assert str[i..][0] == str[i];
      assert str[i..][1..] == str[i + 1..];
      assert remove_word(str[i..]) == remove_word(str[i + 1..]);
      i := i + 1;
    }

    assert i > 0;
    if i < |str| {
      assert str[i] == ' ';
      assert |str[i..]| > 0;
      assert str[i..][0] == ' ';
      assert remove_word(str[i..]) == str[i..];
    } else {
      assert i == |str|;
      assert str[i..] == "";
      assert remove_word(str[i..]) == "";
    }
    assert remove_word(str) == str[i..];

    var rest := p_5_7_count_words(str[i..]);
    word_count := 1 + rest;
    assert remove_leading_spaces(str) == str;
    assert count_words(str) == 1 + count_words(remove_word(str));
    assert count_words(str) == 1 + count_words(str[i..]);
  }
}
```

```
}
```

Final Program

```
function remove_leading_spaces(s: string): (result: string)
  ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] == ' ' then remove_leading_spaces(s[1..])
  else s
}

function remove_word(s: string): (result: string)
  ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] != ' ' then remove_word(s[1..])
  else s
}

function count_words(s: string): int
  decreases |s|
{
  var s2 := remove_leading_spaces(s);
  if |s2| > 0 then 1 + count_words(remove_word(s2))
  else 0
}

method p_5_7_count_words(str: string) returns (word_count: int)
  ensures word_count == count_words(str)
{
  if |str| == 0 {
    word_count := 0;
    assert remove_leading_spaces(str) == str;
    assert count_words(str) == 0;
  } else if str[0] == ' ' {
    var rest := p_5_7_count_words(str[1..]);
    word_count := rest;
    assert str[1..] == str[1..];
    assert remove_leading_spaces(str) == remove_leading_spaces(str[1..]);
    assert count_words(str) == count_words(str[1..]);
  } else {
    var i := 0;
    while i < |str| && str[i] != ' '
      invariant 0 <= i <= |str|
      invariant remove_word(str) == remove_word(str[i..])
    {
      assert |str[i..]| > 0;
      assert str[i..][0] == str[i];
      assert str[i..][1..] == str[i + 1..];
    }
  }
}
```

```

    assert remove_word(str[i..]) == remove_word(str[i + 1..]);
    i := i + 1;
}

assert i > 0;
if i < |str| {
    assert str[i] == ' ';
    assert |str[i..]| > 0;
    assert str[i..][0] == ' ';
    assert remove_word(str[i..]) == str[i..];
} else {
    assert i == |str|;
    assert str[i..] == "";
    assert remove_word(str[i..]) == "";
}
assert remove_word(str) == str[i..];

var rest := p_5_7_count_words(str[i..]);
word_count := 1 + rest;
assert remove_leading_spaces(str) == str;
assert count_words(str) == 1 + count_words(remove_word(str));
assert count_words(str) == 1 + count_words(str[i..]);
}
}

```

Total Token Usage

Input tokens: 748

Output tokens: 3587

Reasoning tokens: 3072

Sum of 'total tokens': 4335

Experiment Timings

Overall Experiment started at 1780678925736, ended at 1780679051566, lasting 125830ms (125.83 seconds)

Iteration #1 started at 1780678925742, ended at 1780679051566, lasting 125824ms (125.82 seconds)

Experiment 'p_5_9_a_sphere_volume' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_a_sphere_volume

Natural language statement: Write a method that computes the volume of a sphere given its radius.

Method signature: p_5_9_a_sphere_volume(r: real) returns (volume: real)

Ensures

- `volume == (4.0 / 3.0) * 3.14159 * r * r * r`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that computes the volume of a sphere given its radius.

The signature should be:

`p_5_9_a_sphere_volume(r: real) returns (volume: real)`

The method should respect the following contract:

`ensures volume == (4.0 / 3.0) * 3.14159 * r * r * r`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmzHPVbYwRASvOFIovKzBnTjz1EZ6

Model: gpt-5.5-2026-04-23

Created at: 1780567675

Finish reason: stop

Usage: 296 tokens in, and 134 tokens out

```
{  
  volume := (4.0 / 3.0) * 3.14159 * r * r * r;  
}
```

Final Program

```
method p_5_9_a_sphere_volume(r: real) returns (volume: real)  
  ensures volume == (4.0 / 3.0) * 3.14159 * r * r * r  
{  
  volume := (4.0 / 3.0) * 3.14159 * r * r * r;  
}
```

Total Token Usage

Input tokens: 296

Output tokens: 134

Reasoning tokens: 87

Sum of ‘total tokens’: 430

Experiment Timings

Overall Experiment started at 1780567674436, ended at 1780567715461, lasting 41025ms (41.03 seconds)

Iteration #1 started at 1780567674437, ended at 1780567715461, lasting 41024ms (41.02 seconds)

Experiment 'p_5_9_b_sphere_surface' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_b_sphere_surface

Natural language statement: Write a method that computes the surface of a sphere given its radius.

Method signature: p_5_9_b_sphere_surface(r: real) returns (surface: real)

Ensures

- `surface == 4.0 * 3.14159 * r * r`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that computes the surface of a sphere given its radius.

The signature should be:

`p_5_9_b_sphere_surface(r: real) returns (surface: real)`

The method should respect the following contract:

`ensures surface == 4.0 * 3.14159 * r * r`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmzI9vRsVaG896J8ve2CDGPfPhwdm

Model: gpt-5.5-2026-04-23

Created at: 1780567721

Finish reason: stop

Usage: 280 tokens in, and 116 tokens out

```
{  
  surface := 4.0 * 3.14159 * r * r;  
}
```

Final Program

```
method p_5_9_b_sphere_surface(r: real) returns (surface: real)  
  ensures surface == 4.0 * 3.14159 * r * r  
{  
  surface := 4.0 * 3.14159 * r * r;  
}
```

Total Token Usage

Input tokens: 280

Output tokens: 116

Reasoning tokens: 77

Sum of ‘total tokens’: 396

Experiment Timings

Overall Experiment started at 1780567721224, ended at 1780567750824, lasting 29600ms (29.60 seconds)

Iteration #1 started at 1780567721226, ended at 1780567750824, lasting 29598ms (29.60 seconds)

Experiment ‘p_5_9_c_cylinder_volume’ Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_c_cylinder_volume

Natural language statement: Write a method that computes the volume of a cylinder with a circular base, given radius and height.

Method signature: p_5_9_c_cylinder_volume(r: real, h: real) returns (volume: real)

Ensures

- `volume == 3.14159 * r * r * h`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that computes the volume of a cylinder with a circular base, given radius
↪ and height.

The signature should be:

`p_5_9_c_cylinder_volume(r: real, h: real) returns (volume: real)`

The method should respect the following contract:

`ensures volume == 3.14159 * r * r * h`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmzIeYCYmMc1YVfLl0WmmBDQWVmzr

Model: gpt-5.5-2026-04-23

Created at: 1780567752

Finish reason: stop

Usage: 294 tokens in, and 117 tokens out

```
{  
  volume := 3.14159 * r * r * h;  
}
```

Final Program

```
method p_5_9_c_cylinder_volume(r: real, h: real) returns (volume: real)  
  ensures volume == 3.14159 * r * r * h  
{  
  volume := 3.14159 * r * r * h;  
}
```

Total Token Usage

Input tokens: 294

Output tokens: 117

Reasoning tokens: 81

Sum of ‘total tokens’: 411

Experiment Timings

Overall Experiment started at 1780567752187, ended at 1780567785926, lasting 33739ms (33.74 seconds)

Iteration #1 started at 1780567752187, ended at 1780567785926, lasting 33739ms (33.74 seconds)

Experiment 'p_5_9_d_cylinder_surface' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_d_cylinder_surface

Natural language statement: Write a method that computes the surface of a cylinder with a circular base, given radius and height.

Method signature: p_5_9_d_cylinder_surface(r: real, h: real) returns (surface: real)

Ensures

- `surface == 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that computes the surface of a cylinder with a circular base, given radius
→ and height.

The signature should be:

`p_5_9_d_cylinder_surface(r: real, h: real) returns (surface: real)`

The method should respect the following contract:

`ensures surface == 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4ZrvV9CBp35T1ZCIMu93CCg2dQP

Model: gpt-5.5-2026-04-23

Created at: 1780588039

Finish reason: stop

Usage: 330 tokens in, and 128 tokens out

```
{  
  surface := 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r;  
}
```

Final Program

```
method p_5_9_d_cylinder_surface(r: real, h: real) returns (surface: real)  
  ensures surface == 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r  
{  
  surface := 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r;  
}
```

Total Token Usage

Input tokens: 330

Output tokens: 128

Reasoning tokens: 74

Sum of ‘total tokens’: 458

Experiment Timings

Overall Experiment started at 1780588039091, ended at 1780588058060, lasting 18969ms (18.97 seconds)

Iteration #1 started at 1780588039093, ended at 1780588058057, lasting 18964ms (18.96 seconds)

Experiment 'p_5_9_e_cone_volume' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_e_cone_volume

Natural language statement: Write a method that computes the volume of a cone with a circular base, given radius and height.

Method signature: p_5_9_e_cone_volume(r: real, h: real) returns (volume: real)

Ensures

- `volume == (1.0 / 3.0) * 3.14159 * r * r * h`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that computes the volume of a cone with a circular base, given radius and
↪ height.

The signature should be:

`p_5_9_e_cone_volume(r: real, h: real) returns (volume: real)`

The method should respect the following contract:

`ensures volume == (1.0 / 3.0) * 3.14159 * r * r * h`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmzTAOrvFnL0IA77L0yYcqnd1QLeK

Model: gpt-5.5-2026-04-23

Created at: 1780568404

Finish reason: stop

Usage: 316 tokens in, and 157 tokens out

```
{  
  volume := (1.0 / 3.0) * 3.14159 * r * r * h;  
}
```

Final Program

```
method p_5_9_e_cone_volume(r: real, h: real) returns (volume: real)  
  ensures volume == (1.0 / 3.0) * 3.14159 * r * r * h  
{  
  volume := (1.0 / 3.0) * 3.14159 * r * r * h;  
}
```

Total Token Usage

Input tokens: 316

Output tokens: 157

Reasoning tokens: 110

Sum of ‘total tokens’: 473

Experiment Timings

Overall Experiment started at 1780568399659, ended at 1780568470177, lasting 70518ms (70.52 seconds)

Iteration #1 started at 1780568399660, ended at 1780568470177, lasting 70517ms (70.52 seconds)

